

Daniel Kwan

(604) 352-4858 | daniel.kwan1@uwaterloo.ca | daniel.uwce.ca | github.com/danielkwan-dev | linkedin.com/in/daniel-kwan

EDUCATION

University of Waterloo

Sep 2025 – Present

Bachelor of Applied Science, Computer Engineering, Honours, Co-op

Waterloo, ON

TECHNICAL SKILLS

Languages: C, C++, Python, Go, TypeScript, JavaScript, SQL

Technologies: FastAPI, Node.js, Flask, React, Next.js, PyTorch, Hugging Face, YOLO, OpenCV, XGBoost

Developer Tools: Docker, Kubernetes, PostgreSQL, Azure, GitHub Actions, AWS, Apigee, SAP, Postman, Linux

EXPERIENCE

Machine Learning Engineer Intern

Jan 2027 - Apr 2027

HubHead

Toronto, ON

- Incoming Winter 2027

Software Developer Intern

May 2026 – Aug 2026

ARITZIA

Vancouver, BC

- Developed **Node.js** scripts that chain calls to OAuth-authenticated **Apigee** and **Azure** REST APIs to automate inventory resets, cutting each from **1 hour** to **6 minutes** and saving **~30 engineering hours** per project
- Built a **Postman** performance-testing suite for **SAP CPI REST iFlows**, executing **500+ requests** to benchmark latency, status codes, and throughput and detect regressions before release
- Automated bulk order creations with **Postman**, cutting daily QA setup time from **~2 hours** to **10 minutes**

Software Engineer

Jan 2026 – Present

Waterloo Aerial Robotics Group

Waterloo, ON

- Developed a MAVLink ground station using a **3-process multiprocessing architecture**, isolating telemetry ingestion from command dispatch so stalled workers cannot block outgoing flight commands
- Sustained **30+ FPS** in an **OpenCV** red and blue target-segmentation pipeline by decoupling frame capture from inference, preventing slow inference cycles from blocking the capture loop
- Reduced heading error to under **5°** by computing relative yaw corrections from live MAVLink telemetry and reissuing commands each control cycle

PROJECTS

🔗 Vulkan Machine Learning Framework | C++20, Vulkan, GLSL, CMake, Python

- Created a **C++20 LLM inference engine** from scratch on **Vulkan**, running **10** architectures (Llama, Qwen, Gemma, Mistral, Phi) on any GPU, by implementing tensors, GPU kernels, BPE tokenizers and Jinja templating
- Outperformed llama.cpp's Vulkan backend in decoding speed on all 6 models benchmarked, by writing int8 dot-product kernels that saturate memory bandwidth (**~50 GB/s**), taking TinyLlama from **28 to 49 tokens/s**
- Ran **7B-parameter** models on an integrated laptop GPU, by loading quantized GGUF weights (Q8_0, Q4_K, Q6_K) and building a sliding-window ring-buffer KV cache that cut Gemma 3's cache memory by **69%**
- Served models to OpenAI clients and lm-evaluation-harness, by building an OpenAI-compatible HTTP server with streaming, tool calling and logprobs, matching transformers' ARC-Easy score exactly

🔗 Exoplanet Machine Learning Identifier (NASA Space Apps) | Python, XGBoost, scikit-learn, FastAPI, React, SHAP

- Placed **2nd** out of **150+ projects** in the highest-difficulty challenge of the NASA Space Apps Challenge
- Rebuilt a leakage-safe exoplanet classifier by quarantining target-derived features and holding out entire host stars, achieving **77% zero-shot accuracy** on **2,562 TESS objects** versus a **51% majority baseline**
- Built an end-to-end NASA data pipeline using **11 cross-mission features**, Optuna-tuned **XGBoost**, and **SHAP** to validate model performance across Kepler training data and unseen TESS observations
- Developed a **FastAPI** and **React** application with a **WebGL** sky map for interactive exploration, batch prediction, and model explanations, backed by **156 offline tests** and **GitHub Actions CI**

🔗 Aegis (YaleHack Winner) | Python, PyTorch, FastAPI, DistilBERT, PostgreSQL, Next.js, SQL, YOLOv8

- Built a privacy-intelligence platform using **FastAPI**, **PostgreSQL**, and **Next.js**, fine-tuning DistilBERT NER to **0.851 entity F1** on **6,000** weakly labeled samples for location, time, business, and activity extraction